
Conception et Développement d'un Écosystème Académique Numérique Intégré : EMSIGHT

AMJAD AHRRAR
HAJAR CHABLI
NIZAR BTIRA
NIZAR EL IDRYSY
OWAIS BAKKALI

3^{ÈME} ANNÉE — INGÉNIERIE INFORMATIQUE ET RÉSEAUX
(3IIRG3)
EMSI TANGER
ANNÉE UNIVERSITAIRE 2025–2026

2025 – 2026

Tuteur(s) EMSI :
EL KAYSOUNI WAFAE
Membres du jury :
EL KAYSOUNI WAFAE
ELUAHHABI ZAKIA

Dédicace

*À nos familles, pour leur soutien indéfectible et leurs sacrifices
silencieux
tout au long de ce parcours académique.*

*À nos professeurs, pour la transmission du savoir,
la rigueur qu'ils nous ont inculquée et la confiance qu'ils nous ont
accordée.*

*À Mme. EL KAYSOUNI WAF AE, notre encadrante,
dont les conseils bienveillants et la disponibilité ont guidé chaque
étape de ce projet.*

*À chaque étudiant EMSI qui, demain,
utilisera EMSIGHT comme outil du quotidien.*

Résumé

EMSIGHT (*EMSI – Insight*) est une plateforme numérique académique et professionnelle unifiée conçue pour la communauté de l'École Marocaine des Sciences de l'Ingénieur. Ce projet répond à une problématique concrète et documentée : la fragmentation des outils numériques utilisés au quotidien par les étudiants, les enseignants et les administrateurs, entraînant une perte d'efficacité et une expérience utilisateur dégradée.

La solution proposée regroupe trois modules complémentaires et interdépendants : **EMSIGHT Portal** pour la gestion académique centralisée (notes, absences, emplois du temps hebdomadaires, bulletins scolaires, demandes de documents administratifs et notifications) ; **EMSI Share** pour le partage collaboratif et validé de ressources pédagogiques entre pairs ; et **EMSI Community** pour le réseau professionnel et l'aide à l'insertion via un analyseur de CV propulsé par intelligence artificielle.

Sur le plan technique, EMSIGHT repose sur une architecture décuplée client-serveur : un frontend **React.js** (Vite 8, React 19) et un backend **Django REST Framework** (Python 3, Django 6) communiquant via une API REST. L'analyse intelligente de CV exploite le modèle **Llama 3.3 70B** via l'API Groq, avec un mécanisme de repli heuristique en cas d'indisponibilité du service.

Mots-clés : Écosystème académique numérique, Django REST Framework, React.js, API REST, Intelligence Artificielle, Analyse de CV, Architecture décuplée, RBAC.

Abstract

EMSIGHT (*EMSI – Insight*) is a unified academic and professional digital platform designed for the community of the École Marocaine des Sciences de l’Ingénieur. The project addresses a documented real-world challenge : the fragmentation of digital tools used daily by students, teachers, and administrators, resulting in inefficiency and a degraded user experience.

The proposed solution consolidates three complementary modules : **EMSIGHT Portal** for centralised academic management (grades, attendance, weekly schedules, report cards, administrative document requests, and notifications); **EMSI Share** for collaborative, peer-validated sharing of educational resources; and **EMSI Community** for professional networking and career support, including an AI-powered CV analyser.

Technically, EMSIGHT relies on a decoupled client-server architecture : a **React.js** (Vite 8, React 19) frontend and a **Django REST Framework** (Python 3, Django 6) backend communicating through a REST API. Intelligent CV analysis leverages the **Llama 3.3 70B** model via the Groq API, with a built-in heuristic fallback mechanism.

Keywords : Academic digital ecosystem, Django REST Framework, React.js, REST API, Artificial Intelligence, CV Analysis, Decoupled architecture, RBAC.

Remerciements

Nous tenons à exprimer notre profonde gratitude envers l'ensemble des personnes qui ont contribué, de près ou de loin, à la réalisation de ce Projet de Fin d'Année.

Nos remerciements s'adressent en premier lieu à **l'École Marocaine des Sciences de l'Ingénieur — EMSI Tanger** pour le cadre d'apprentissage exceptionnel qu'elle nous offre, les ressources pédagogiques mises à notre disposition et l'opportunité de mener à bien ce projet ambitieux dans les meilleures conditions.

Un immense merci à notre encadrante pédagogique, **Mme. EL KAYSOUNI WAFAE**, dont le suivi attentif, les conseils avisés et la disponibilité constante ont été déterminants dans l'orientation et la structuration de notre travail tout au long de cette année. Sa rigueur méthodologique et sa passion pour l'enseignement constituent pour nous une source d'inspiration durable.

Nous remercions chaleureusement les **membres du jury** pour le temps et l'attention qu'ils consacrent à l'évaluation de ce travail.

Nos remerciements vont également à nos **familles et proches** pour leur soutien moral constant, leur patience et leur compréhension lors des longues séances de travail, ainsi qu'à nos **collègues de promotion** pour les échanges enrichissants qui ont nourri notre réflexion tout au long de ce projet.

Table des matières

Dédicace	1
Résumé	2
Abstract	3
Introduction Générale	11
1 Étude de l'existant	13
Introduction	13
1.1 Contexte général	13
1.2 Solutions existantes	14
1.3 Limites des solutions actuelles	14
1.3.1 Fragmentation des outils	14
1.3.2 Absence de collaboration pédagogique structurée	15
1.3.3 Absence d'aide à l'insertion professionnelle par IA	15
1.3.4 Interfaces vieillissantes et non responsives	15
1.3.5 Absence de système de notification unifié	15
1.4 Synthèse et justification du projet	15
Conclusion	16
2 Analyse et spécification des besoins	17
Introduction	17
2.1 Identification des acteurs	17
2.2 Besoins fonctionnels	18
2.2.1 Module EMSIGHT Portal — Gestion académique	18

2.2.2	Module EMSI Share — Ressources collaboratives	19
2.2.3	Module EMSI Community — Réseau et carrière	19
2.3	Besoins non fonctionnels	20
2.4	Diagramme des cas d'utilisation	20
	Conclusion	21
3	Conception	22
	Introduction	22
3.1	Architecture du système	22
3.1.1	Architecture décuplée client-serveur	22
3.1.2	Organisation du code source	23
3.2	Modélisation	24
3.2.1	Diagramme de classes	24
3.2.2	Diagramme d'activité	25
3.2.3	Modèle Conceptuel de Données (MCD)	26
3.2.4	Modèle Logique de Données (MLD)	27
	Conclusion	28
4	Implémentation et réalisation	29
	Introduction	29
4.1	Environnement de développement	29
4.2	Technologies utilisées	30
4.2.1	Backend : Django et Django REST Framework	30
4.2.2	Frontend : React.js et Vite	30
4.2.3	Intelligence artificielle — API Groq + fallback heuristique	30
4.2.4	Base de données	31
4.3	Interfaces utilisateur	31
4.3.1	Page de connexion	31
4.3.2	Dashboard étudiant — EMSIGHT Portal	31
4.3.3	Module EMSI Share — Bibliothèque collaborative	32
4.3.4	Analyseur de CV — EMSI Community	33
4.3.5	Dashboard administrateur — Gestion des emplois du temps	34
4.4	Extraits de code significatifs	34
4.4.1	Modèle CustomUser — accounts/models.py	34
4.4.2	Soumission groupée des notes — portal/views.py	35

4.4.3 Routage par rôle — src/App.jsx	36
Conclusion	36
Conclusion Générale	38
Bilan du projet	38
Apports du projet	38
Perspectives	39
A Extraits de Code Significatifs	41
A.1 Backend – Modèle CustomUser (accounts/models.py)	41
A.2 Backend – Endpoint de Connexion (accounts/views.py)	42
A.3 Backend – Service Analyse CV avec Fallback (community/ai_service.py)	43
A.4 Frontend – Client Axios Centralisé (src/api.js)	44
A.5 Frontend – Routage par Rôle (src/App.jsx, extrait)	44
A.6 Backend – Soumission Groupée des Notes (portal/views.py, extrait)	45
B Captures d'Écran Complémentaires	47
B.1 Module Community – Fil d'Actualité	47
B.2 Dashboard Professeur – Saisie des Notes	48
B.3 Bulletin Étudiant – Vue Transcript Global	48

Table des figures

2.1	Diagramme des cas d'utilisation global — Plateforme EMSIGHT . . .	21
3.1	Architecture générale de la plateforme EMSIGHT	23
3.2	Diagramme de classes — hiérarchie des utilisateurs et entités métier EMSIGHT	25
3.3	Diagramme d'activité — Flux utilisateur complet d'EMSIGHT	26
3.4	Modèle Conceptuel de Données (MCD) — EMSIGHT	27
4.1	Interface de connexion — authentification centralisée EMSIGHT . . .	31
4.2	Dashboard principal d'un étudiant — Vue d'ensemble du portail aca- démique EMSIGHT	32
4.3	Module EMSI Share — bibliothèque collaborative avec badges de validation multi-enseignants	33
4.4	Interface de l'analyseur de CV IA — score de correspondance et re- commandations personnalisées	33
4.5	Outil de gestion des emplois du temps — grille interactive hebdoma- daire par classe	34
B.1	Fil d'actualité EMSI Community : publication de posts avec médias, badges de rôle (Admin, Teacher) et système de validation par les enseignants	47
B.2	Interface de saisie des notes par classe – Dashboard Professeur : sé- lection du module, de la classe, du semestre et du type d'évaluation, puis saisie individuelle avec soumission groupée	48

B.3 Relevé de notes global (Transcript) – Dashboard Étudiant : vue multi-années avec détail par semestre, moyenne générale, statut de validation et rattrapages	48
---	----

Liste des tableaux

1.1	Inventaire des outils numériques existants à l'EMSI	14
1.2	Synthèse des limites et réponses apportées par EMSIGHT	16
2.1	Synthèse des besoins non-fonctionnels d'EMSIGHT	20
4.1	Environnement de développement et outils utilisés	29

Introduction Générale

La transformation numérique de l’enseignement supérieur est désormais une réalité incontournable. À l’échelle mondiale, les établissements d’enseignement adoptent massivement des outils digitaux pour améliorer la qualité du suivi pédagogique, faciliter la communication entre acteurs académiques et préparer les étudiants à un marché du travail de plus en plus exigeant.

L’**École Marocaine des Sciences de l’Ingénieur (EMSI)**, établissement de référence dans la formation d’ingénieurs au Maroc, dispose d’une communauté académique composée d’étudiants, d’enseignants et de personnels administratifs qui interagit quotidiennement avec un ensemble de services numériques — consultation des notes, gestion des absences, partage de ressources pédagogiques, publication d’offres professionnelles, etc. Cependant, ces services sont aujourd’hui dispersés sur plusieurs plateformes distinctes, sans cohérence visuelle ni interconnexion fonctionnelle.

C’est dans ce contexte que le projet **EMSIGHT** (*EMSI — Insight*) prend tout son sens. Il vise à concevoir et développer un *écosystème numérique unifié*, où l’ensemble des services académiques et professionnels essentiels coexistent sous une interface unique, accessible et moderne, avec une authentification centralisée et une expérience utilisateur adaptée à chaque rôle (étudiant, enseignant, administrateur).

La plateforme est structurée autour de trois modules complémentaires : **EMSIGHT Portal** pour la gestion académique, **EMSI Share** pour le partage collaboratif et validé de ressources pédagogiques entre pairs, et **EMSI Community** pour le réseau professionnel et l’aide à l’insertion via un analyseur de CV propulsé par intelligence artificielle.

Ce rapport est organisé en quatre chapitres. Le **Chapitre 1** présente l’étude de

l'existant et la justification du projet. Le **Chapitre 2** expose l'analyse et la spécification des besoins fonctionnels et non-fonctionnels. Le **Chapitre 3** détaille la conception et la modélisation du système. Le **Chapitre 4** décrit l'implémentation technique et présente les interfaces réalisées.

Chapitre 1

Étude de l'existant

Introduction

Ce chapitre présente le contexte général dans lequel s'inscrit le projet EMSIGHT, passe en revue les solutions numériques existantes à l'EMSI et dans le secteur, identifie leurs limites, puis justifie la pertinence d'une nouvelle plateforme intégrée.

1.1 Contexte général

La digitalisation de l'enseignement supérieur marocain s'intensifie depuis plusieurs années, portée par des initiatives telles que le Plan d'Accélération de la Transformation de l'Écosystème de l'Enseignement Supérieur, de la Recherche Scientifique et de l'Innovation (PACTES). Dans ce contexte, les grandes écoles d'ingénierie comme l'EMSI ressentent la nécessité d'outiller leurs communautés avec des plateformes numériques modernes, cohérentes et efficaces.

L'EMSI — École Marocaine des Sciences de l'Ingénieur — accueille chaque année plusieurs milliers d'étudiants répartis sur de nombreux campus au Maroc. Sa communauté académique interagit quotidiennement avec des outils numériques pour consulter les notes, gérer les absences, accéder à des ressources pédagogiques ou rechercher des opportunités professionnelles. Or, ces outils sont actuellement fragmentés entre plusieurs URL et interfaces hétérogènes.

1.2 Solutions existantes

Avant EMSIGHT, la communauté EMSI Tanger disposait de plusieurs outils numériques fonctionnant de manière indépendante, résumés dans le tableau 1.1.

TABLE 1.1 – Inventaire des outils numériques existants à l'EMSI

Plateforme	Fonctionnalité principale	Accès
<code>student.emsi.ma</code>	Portail académique : notes, absences, bulletins	Étudiant
<code>emsicommunity.com</code>	Réseau social, offres de stage et d'emploi	Tous
<i>(aucune)</i>	Partage structuré de ressources pédagogiques entre pairs	—
<i>(email / WhatsApp)</i>	Communication informelle entre acteurs	Tous

À l'échelle internationale, plusieurs solutions constituent des références dans le secteur. **Moodle** est une plateforme d'apprentissage en ligne open-source très répandue dans les universités francophones. **Blackboard** propose des fonctionnalités similaires avec une orientation plus américaine. **LinkedIn Learning** se positionne sur l'insertion professionnelle. Ces plateformes restent pertinentes dans leur domaine respectif, mais aucune ne couvre simultanément la gestion académique, le partage de ressources validées par les pairs et l'analyse IA de candidatures dans un contexte EMSI [1].

1.3 Limites des solutions actuelles

L'analyse approfondie de l'écosystème existant révèle plusieurs lacunes qui justifient la création d'EMSIGHT.

1.3.1 Fragmentation des outils

La dispersion des services sur plusieurs URL distinctes impose aux utilisateurs des connexions multiples, des interfaces graphiques hétérogènes et une expérience incohérente. Un étudiant souhaitant consulter ses notes, signaler une absence, consulter

une offre de stage et partager un cours doit naviguer entre trois ou quatre applications différentes, sans transfert de données entre elles.

1.3.2 Absence de collaboration pédagogique structurée

Il n'existe aucune plateforme officielle permettant aux étudiants de déposer leurs supports de cours, exercices ou corrigés, avec un mécanisme de validation qualitative par les enseignants. Le partage se fait de manière informelle, via des groupes WhatsApp ou des drives personnels, sans traçabilité ni garantie d'exactitude.

1.3.3 Absence d'aide à l'insertion professionnelle par IA

La plateforme communautaire existante propose des offres d'emploi et de stage mais ne fournit aucun outil permettant à l'étudiant d'évaluer la pertinence de son profil par rapport à une offre. L'absence d'un analyseur de CV intelligent prive les étudiants d'un accompagnement personnalisé dans leur transition vers le marché du travail.

1.3.4 Interfaces vieillissantes et non responsives

Les portails existants présentent des interfaces non optimisées pour les écrans mobiles, sans support du mode sombre, et avec des temps de chargement importants liés à des architectures web traditionnelles sans application monopage (SPA).

1.3.5 Absence de système de notification unifié

Les communications entre enseignants, administrateurs et étudiants passent par des canaux disparates (email, WhatsApp, annonces orales). Il n'existe pas de système de notification numérique centralisé et persistant au sein du même portail académique.

1.4 Synthèse et justification du projet

Face aux limites documentées ci-dessus, EMSIGHT se justifie par la nécessité de proposer une rupture technologique maîtrisée : remplacer la mosaïque d'outils fragmentés par un écosystème cohérent.

TABLE 1.2 – Synthèse des limites et réponses apportées par EMSIGHT

Limite identifiée	Réponse EMSIGHT
Connexions multiples / fragmentation	Authentification unique — un seul compte pour tout
Pas de partage structuré de ressources	Module EMSI Share avec validation multi-enseignants
Pas d'aide IA à l'insertion	Analyseur de CV IA (Llama 3.3 70B via Groq API)
Interfaces vieillissantes / non responsives	SPA React avec design responsive, thème clair/sombre
Notifications disparates	Système de notifications centralisé et persistant
Aucune gestion numérique des emplois du temps	Emplois du temps JSON dynamiques par classe

Conclusion

Ce chapitre a mis en évidence les lacunes des outils numériques actuels de l'EMSI et justifié la conception d'EMSIGHT comme solution intégrée. Le chapitre suivant spécifie en détail les besoins fonctionnels et non-fonctionnels du système.

Chapitre 2

Analyse et spécification des besoins

Introduction

Ce chapitre identifie les acteurs du système EMSIGHT, détaille les besoins fonctionnels de chaque module, présente les exigences non-fonctionnelles transversales, et introduit le diagramme des cas d'utilisation global.

2.1 Identification des acteurs

Le système EMSIGHT implique trois acteurs principaux :

1. **L'Étudiant** : il consulte ses notes, absences et bulletins ; soumet des justifications d'absence ; partage et télécharge des ressources pédagogiques ; consulte les offres professionnelles ; soumet son CV pour analyse IA ; reçoit des notifications et effectue des demandes de documents administratifs.
2. **Le Professeur** : il saisit les notes et gère les absences ; valide les ressources partagées via un badge de certification ; publie des événements et envoie des notifications ciblées.
3. **L'Administrateur** : il gère le répertoire des utilisateurs, publie les emplois du temps, valide les demandes de documents, et modère l'ensemble des contenus de la plateforme.

2.2 Besoins fonctionnels

2.2.1 Module EMSIGHT Portal — Gestion académique

Fonctionnalités côté Étudiant

- Consultation des notes par matière, semestre et année académique, avec calcul automatique de la moyenne générale.
- Visualisation d'un relevé de notes global (transcript) couvrant l'ensemble des années d'études, avec indication du statut de validation (reçu / ajourné).
- Suivi des absences avec indication du statut de justification (en attente, validée, rejetée).
- Soumission de justifications d'absence directement depuis la plateforme.
- Consultation de l'emploi du temps hebdomadaire de sa classe sous forme de grille visuelle à 5 colonnes (lundi-vendredi).
- Réception et suppression de notifications académiques.
- Demande de documents administratifs (attestation de scolarité, attestation de réussite) avec suivi du statut de traitement.

Fonctionnalités côté Professeur

- Saisie et modification des notes par classe et par module assigné.
- Soumission groupée (*bulk submit*) des notes pour toute une classe en une seule opération, avec recalcul automatique de la moyenne générale.
- Gestion des absences : marquage de présence/absence par séance.
- Création, modification et suppression d'événements dans le calendrier académique, ciblés par classe.
- Envoi de notifications à l'ensemble des étudiants de ses classes.
- Visualisation des emplois du temps des classes dont il est responsable.

Fonctionnalités côté Administrateur

- Construction et publication des emplois du temps hebdomadaires par classe via une grille interactive (ajout, suppression, sauvegarde de créneaux).
- Gestion complète du répertoire des utilisateurs : création avec génération automatique d'email institutionnel, assignation de modules et de classes aux

enseignants, suppression de comptes.

- Validation ou rejet des justifications d'absence soumises par les étudiants.
- Traitement (validation / rejet) des demandes de documents administratifs.
- Envoi d'annonces globales à l'ensemble des étudiants.
- Supervision et modification du calendrier académique global.

2.2.2 Module EMSI Share — Ressources collaboratives

- Dépôt de ressources pédagogiques catégorisées : cours, exercices sans correction, exercices avec correction. Formats acceptés : PDF, DOC, DOCX.
- Recherche filtrée par titre, matière et type de ressource.
- Lecture directe des fichiers depuis la plateforme (lien de visualisation).
- **Système de validation multi-enseignants** : tout professeur peut apposer un badge de vérification sur une ressource ; le nombre de validateurs est affiché publiquement avec leurs noms.
- Signalement de ressources inappropriées ou inexactes avec saisie d'un motif.
- Gestion d'une liste de favoris personnelle (*toggle favori*).
- Supervision administrative : consultation des ressources signalées, suppression de contenus problématiques.

2.2.3 Module EMSI Community — Réseau et carrière

- Fil d'actualité social : publication de posts textuels ou avec pièces jointes (images, documents), modification et suppression par l'auteur, validation par les enseignants (badge « Vérifié »).
- Gestion d'événements communautaires (création, modification, suppression par les administrateurs).
- Publication d'offres de stage et d'emploi avec filtres par type (stage, temps plein, temps partiel, freelance) et date d'expiration.
- **Analyseur de CV par Intelligence Artificielle** : l'étudiant télécharge son CV (PDF ou texte), sélectionne une offre cible, et obtient un score de correspondance (0–100) ainsi que des recommandations personnalisées.
- Historique consultable des analyses CV réalisées par l'utilisateur.

2.3 Besoins non fonctionnels

TABLE 2.1 – Synthèse des besoins non-fonctionnels d'EMSIGHT

Critère	Exigence retenue
Sécurité	Authentification par email/mot de passe, contrôle d'accès basé sur les rôles (RBAC) à chaque endpoint, secrets stockés dans des variables d'environnement (.env)
Performance	Rendu SPA sans rechargement complet, requêtes asynchrones via Axios, mises à jour optimistes de l'interface
Disponibilité	Mécanisme de dégradation gracieuse : fallback heuristique Python si l'API IA Groq est indisponible
Maintenabilité	Architecture modulaire : 4 applications Django indépendantes, composants React séparés par rôle et par module
Accessibilité	Design entièrement responsive (Flexbox + CSS Grid), thèmes clair et sombre via CSS Variables
Scalabilité	SQLite en développement, migration transparente vers PostgreSQL en production via l'ORM Django

2.4 Diagramme des cas d'utilisation

La figure 2.1 présente le diagramme des cas d'utilisation global de la plateforme EMSIGHT. Ce diagramme illustre les interactions entre les trois acteurs identifiés et les fonctionnalités organisées par module. Les relations d'inclusion permettent de préciser les cas conditionnels : la validation par les enseignants dans EMSI Share, la soumission groupée des notes dans EMSIGHT Portal, et l'appel à l'API IA dans EMSI Community.

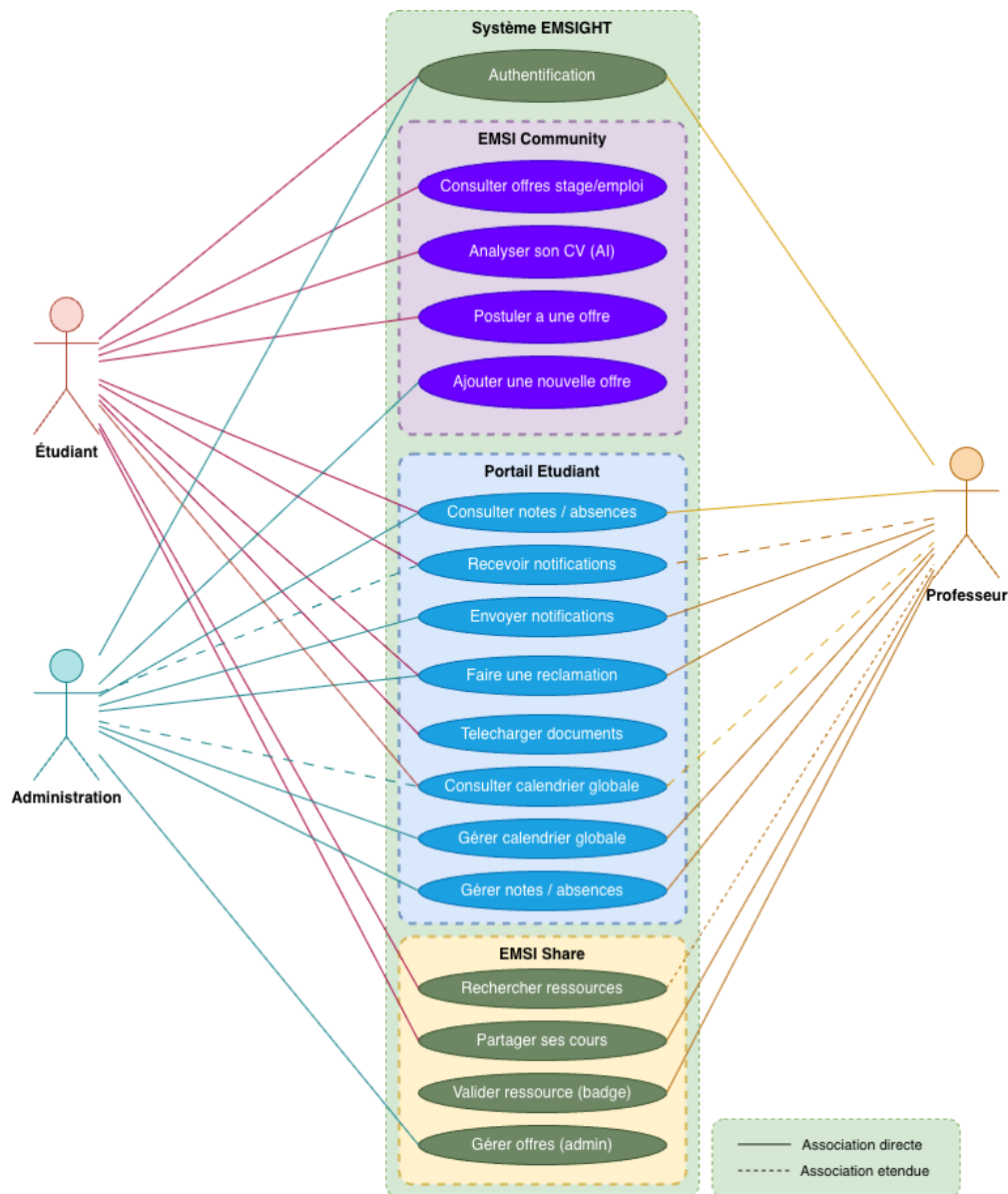


FIGURE 2.1 – Diagramme des cas d'utilisation global — Plateforme EMSIGHT

Conclusion

Ce chapitre a établi de manière exhaustive les besoins fonctionnels et non-fonctionnels d'EMSIGHT, organisés par acteur et par module. La prochaine étape consiste à traduire ces spécifications en modèles de conception (UML, MCD/MLD) qui guideront l'implémentation.

Chapitre 3

Conception

Introduction

Ce chapitre expose les choix architecturaux retenus pour EMSIGHT, présente la modélisation UML (diagrammes de classes, d'activité, de séquence) ainsi que la modélisation des données (MCD et MLD) qui fondent l'implémentation du système.

3.1 Architecture du système

3.1.1 Architecture décuplée client-serveur

EMSIGHT repose sur une **architecture décuplée client-serveur** (*decoupled architecture*), séparant strictement la couche de présentation (frontend React.js) de la couche métier et d'accès aux données (backend Django REST Framework). Cette séparation confère plusieurs avantages : évolution indépendante des deux couches, déploiement séparé possible (conteneurs Docker distincts), et facilitation des tests unitaires et d'intégration.

La communication entre le frontend et le backend s'effectue exclusivement via une **API REST**, avec échange de données au format JSON. Le frontend ne contient aucune logique métier : il se contente d'afficher les données reçues et de transmettre les actions de l'utilisateur au backend.

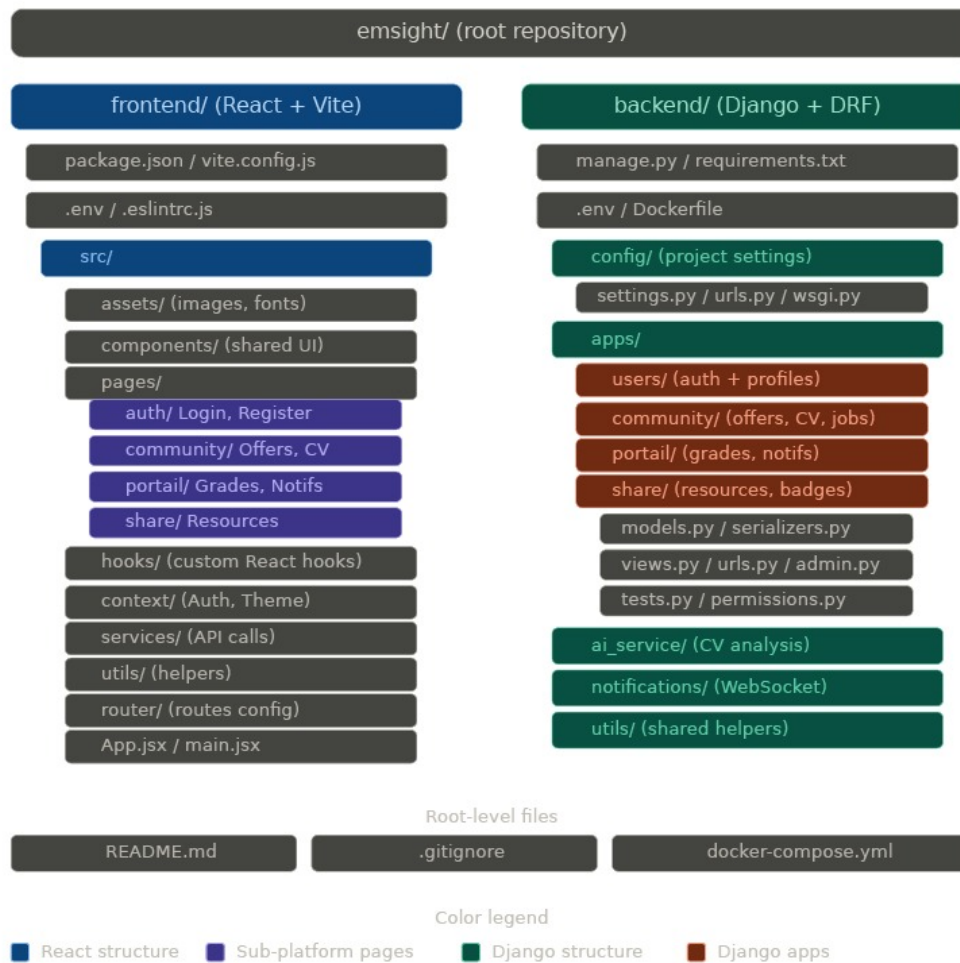


FIGURE 3.1 – Architecture générale de la plateforme EMSIGHT

3.1.2 Organisation du code source

Le dépôt respecte une séparation claire entre les deux couches :

- **Frontend** (frontend/) : application React + Vite, organisée en pages (StudentDashboard, TeacherDashboard, AdminDashboard, CommunityDashboard, ShareDashboard), composants réutilisables (TopBar, Sidebar), et client Axios centralisé (api.js).
- **Backend** (backend/) : projet Django structuré en quatre applications indépendantes — **accounts** (authentification et profils), **portal** (gestion académique), **share** (ressources collaboratives) et **community** (réseau et carrière, dont ai_service.py pour l'analyse IA de CV).

3.2 Modélisation

3.2.1 Diagramme de classes

Le diagramme de classes (figure 3.2) expose la structure orientée objet du système. La classe centrale **CustomUser** (qui étend **AbstractUser** de Django) est spécialisée par composition en **StudentProfile**, **TeacherProfile** et **AdminProfile** via des relations One-to-One.

Les entités métier gravitent autour du modèle utilisateur :

- Application portal : **CalendarEvent**, **Notification**, **ReportCard**, **Grade**, **Absence**, **DocumentRequest**, **ClassSchedule**.
- Application share : **Resource** (avec champ Many-to-Many **validated_by**), **ResourceFavorite**, **ResourceReport**.
- Application community : **Post** (avec **validated_by**), **Event**, **JobOffer**, **CVA-analysis**.

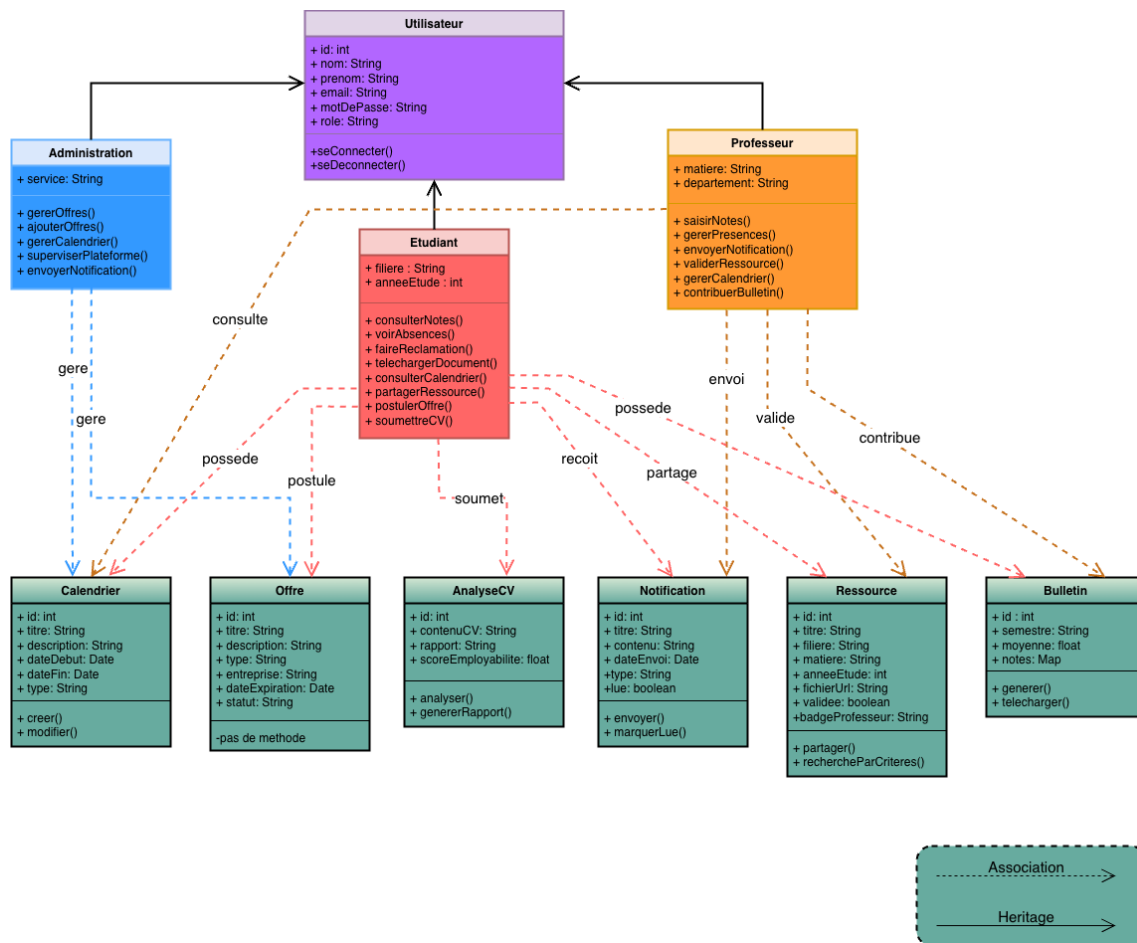


FIGURE 3.2 – Diagramme de classes — hiérarchie des utilisateurs et entités métier EMSIGHT

3.2.2 Diagramme d'activité

Le diagramme d'activité (figure 3.3) illustre le flux de travail global de l'application, depuis l'authentification jusqu'à l'utilisation des modules. Trois scénarios sont modélisés : accès réussi avec routage selon le rôle ; identifiants incorrects avec retour au formulaire ; et accès aux trois modules (Portal, Share, Community) depuis la barre de navigation supérieure.

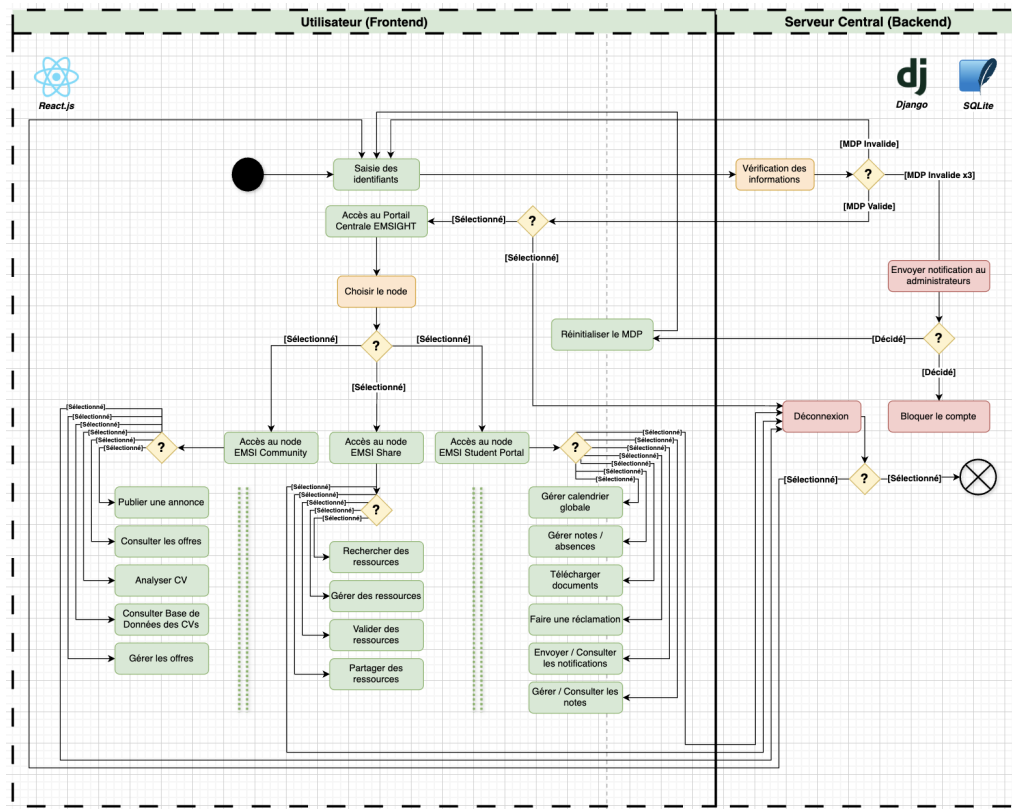


FIGURE 3.3 – Diagramme d'activité — Flux utilisateur complet d'EMSIGHT

3.2.3 Modèle Conceptuel de Données (MCD)

Le MCD (figure 3.4) définit l'organisation logique des données et les relations entre les entités, indépendamment de toute considération d'implémentation.

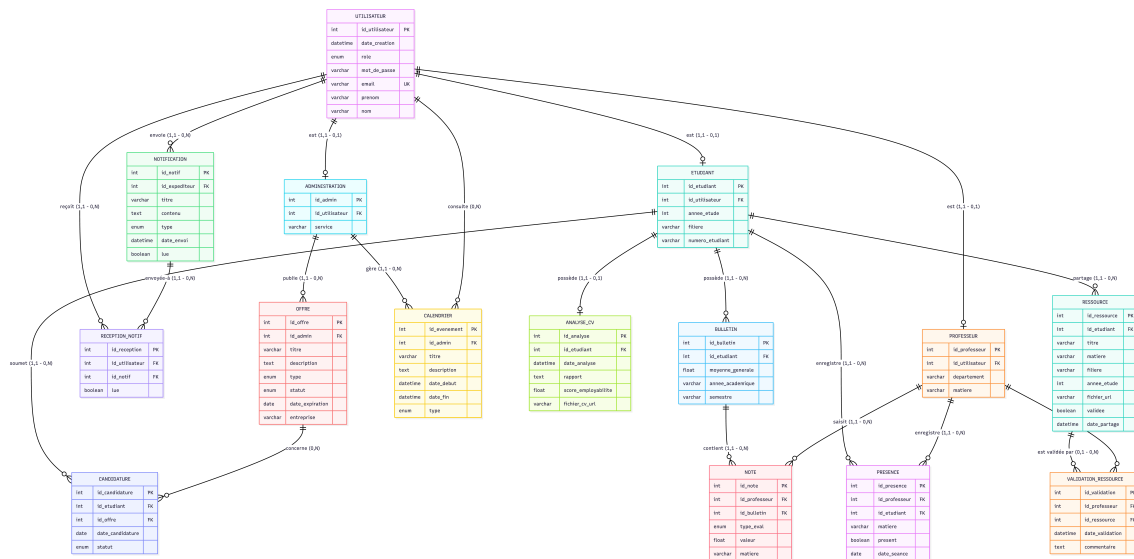


FIGURE 3.4 – Modèle Conceptuel de Données (MCD) — EMSIGHT

3.2.4 Modèle Logique de Données (MLD)

La traduction du MCD en MLD produit les tables relationnelles principales suivantes (clé primaire soulignée, clé étrangère en italique) :

- **accounts_customuser**(id, username, email, password, first_name, last_name, role, matricule, gender, profile_picture)
- **accounts_studentprofile**(id, *user_id*, filiere, annee_etude, numero_etudiant, tutor_name)
- **accounts_teacherprofile**(id, *user_id*, departement, matiere, classes)
- **portal_reportcard**(id, *student_id*, academic_year, semester, general_average)
- **portal_grade**(id, *report_card_id*, subject, evaluation_type, value, is_rattrapage)
- **portal_absence**(id, *student_id*, *teacher_id*, subject, date_seance, is_present, justification_text, justification_status)
- **portal_classschedule**(id, target_class, schedule_data)
- **share_resource**(id, *author_id*, title, subject, resource_type, file, created_at)
- **share_resource_validated_by**(*resource_id*, *customuser_id*) [table de jointure M2M]
- **community_joboffer**(id, *author_id*, title, company, description, requirements, location, job_type, expiration_date)

— **community_cvanalysis**(*id*, *user_id*, *job_offer_id*, *cv_text*, *score*, *suggestions*, *created_at*)

Conclusion

Ce chapitre a posé les bases conceptuelles d'EMSIGHT à travers l'architecture décuplée, les diagrammes UML et la modélisation des données. Le chapitre suivant détaille la mise en œuvre technique et présente les interfaces réalisées.

Chapitre 4

Implémentation et réalisation

Introduction

Ce chapitre présente l'environnement de développement, justifie les choix technologiques, décrit l'implémentation des fonctionnalités clés, et présente les principales interfaces réalisées.

4.1 Environnement de développement

TABLE 4.1 – Environnement de développement et outils utilisés

Outil	Rôle	Version
Visual Studio Code	Éditeur principal (frontend et backend)	1.89+
Git + GitHub	Versionnement, branches de feature, pull requests	2.44+
Postman	Test et documentation des endpoints API REST	11+
draw.io	Modélisation UML (use case, classes, activité)	Web
DB Browser for SQLite	Inspection et gestion de la base de données	3.12+
Node.js	Environnement d'exécution pour le frontend	22+
Python	Langage du backend	3.11+

4.2 Technologies utilisées

4.2.1 Backend : Django et Django REST Framework

Django (version 6.0) est le framework web Python retenu pour le backend. Sa philosophie *batteries included* offre nativement un ORM puissant, un système de migrations, un module d'authentification extensible et une interface d'administration. **Django REST Framework** (DRF 3.17) étend Django pour exposer des API REST : ses `ModelSerializer` génèrent automatiquement la sérialisation JSON à partir des modèles, et ses `ViewSet` factorisent les opérations CRUD [2]. Le contrôle d'accès basé sur les rôles (RBAC) est implémenté via le champ `role` du modèle `CustomUser`.

4.2.2 Frontend : React.js et Vite

React.js (version 19) est la bibliothèque JavaScript retenue pour l'interface utilisateur. Son modèle de composants réutilisables et son système de state local (`useState`, `useEffect`) permettent de construire des interfaces réactives avec un code maintenable [3]. **Vite** (version 8.0) sert de bundler de développement avec Hot Module Replacement (HMR), offrant des temps de rechargement quasi-instantanés [4]. La communication HTTP est gérée par **Axios** (1.15) via un client centralisé (`api.js`) [5].

4.2.3 Intelligence artificielle — API Groq + fallback heuristique

L'analyseur de CV exploite le modèle **Llama 3.3 70B Versatile** via l'**API Groq** [6], qui propose des inférences ultra-rapides. En cas d'indisponibilité de l'API, un **algorithme heuristique Python** (`ai_service.py`) prend le relais automatiquement : il extrait les mots-clés du CV et des exigences de l'offre, filtre les mots vides, mappe les compétences techniques multi-mots, calcule un ratio de correspondance et génère un retour structuré listant les compétences détectées et manquantes.

4.2.4 Base de données

SQLite est utilisé en développement pour sa simplicité de configuration. L'ORM Django abstrait complètement le moteur : la migration vers **PostgreSQL** en production ne nécessite qu'une modification dans `settings.py`, sans toucher au code applicatif.

4.3 Interfaces utilisateur

4.3.1 Page de connexion

La page de connexion constitue le point d'entrée unique de la plateforme. Elle présente un formulaire avec saisie de l'adresse email institutionnelle et du mot de passe, un retour d'erreur explicite, et un état de chargement visuel. Selon le rôle de l'utilisateur authentifié, `App.jsx` le route vers le dashboard approprié (`StudentDashboard`, `TeacherDashboard` ou `AdminDashboard`).

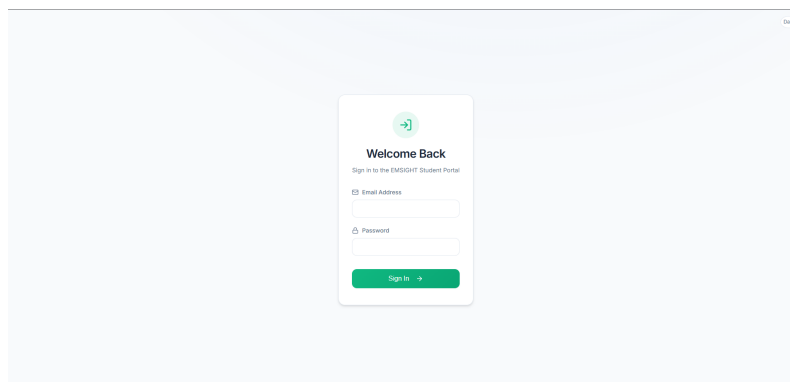


FIGURE 4.1 – Interface de connexion — authentification centralisée EMSIGHT

4.3.2 Dashboard étudiant — EMSIGHT Portal

Le tableau de bord étudiant présente une vue synthétique du profil académique : photo de profil (générée via l'API DiceBear en l'absence de photo personnalisée), informations de profil, statistiques (absences, notifications, demandes de documents), emploi du temps hebdomadaire en grille 5 colonnes et liste des événements à venir filtrés par classe.

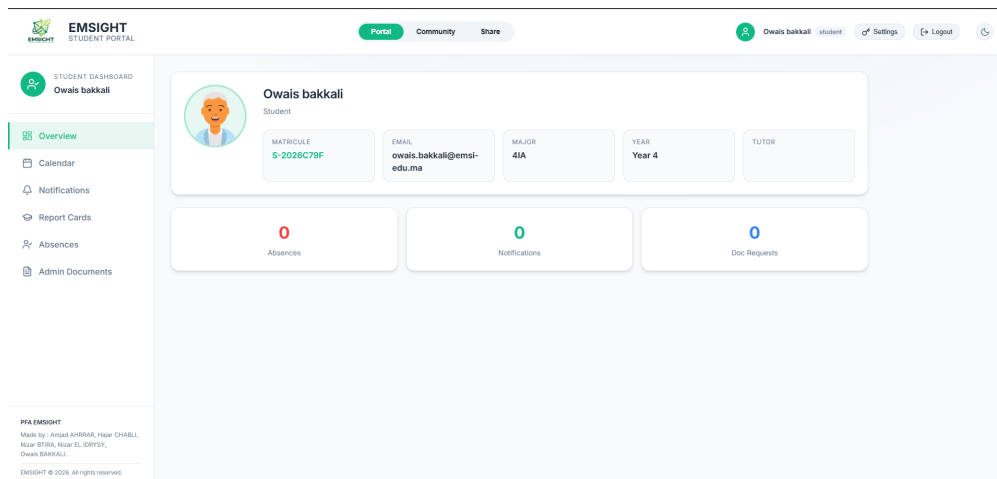


FIGURE 4.2 – Dashboard principal d'un étudiant — Vue d'ensemble du portail académique EMSIGHT

4.3.3 Module EMSI Share — Bibliothèque collaborative

La liste des ressources affiche pour chaque entrée : titre, sujet, type, auteur, nombre de signalements et les badges de validation indiquant les enseignants ayant vérifié le contenu. Des filtres par type et une barre de recherche en temps réel permettent une navigation efficace.

Le champ `validated_by` du modèle `Resource` est une relation Many-to-Many vers `CustomUser`. L'endpoint `POST /api/share/resources/{id}/validate_resource/` bascule la présence de l'enseignant dans cette relation (ajout / retrait). Ce mécanisme de validation décentralisée introduit une dynamique de revue par les pairs inédite à l'EMSI.

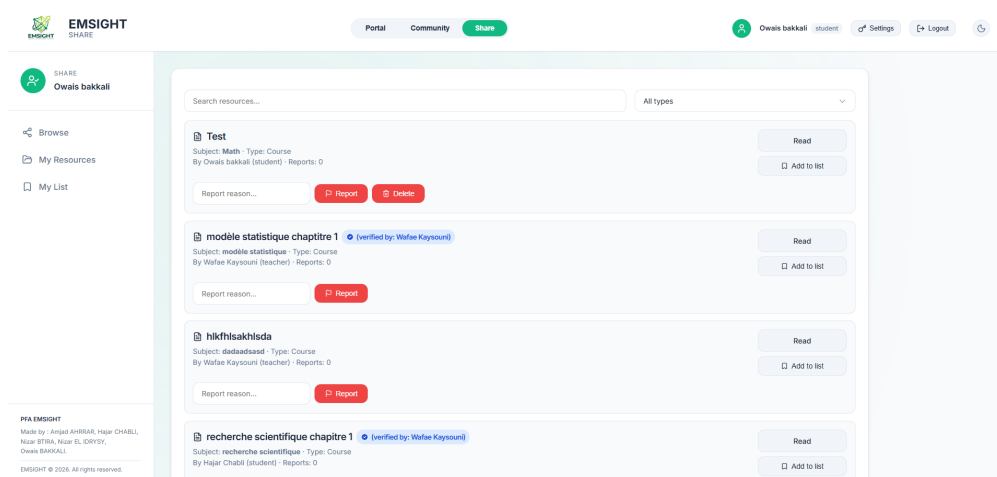


FIGURE 4.3 – Module EMSI Share — bibliothèque collaborative avec badges de validation multi-enseignants

4.3.4 Analyseur de CV — EMSI Community

Le panneau gauche permet à l'étudiant de télécharger son CV (PDF extrait côté navigateur via **pdfjs-dist** [7], ou texte collé manuellement) et de sélectionner une offre cible. Le panneau droit affiche le score de correspondance sous forme circulaire (vert $\geq 70\%$, orange $\geq 40\%$, rouge $< 40\%$) et les recommandations personnalisées.

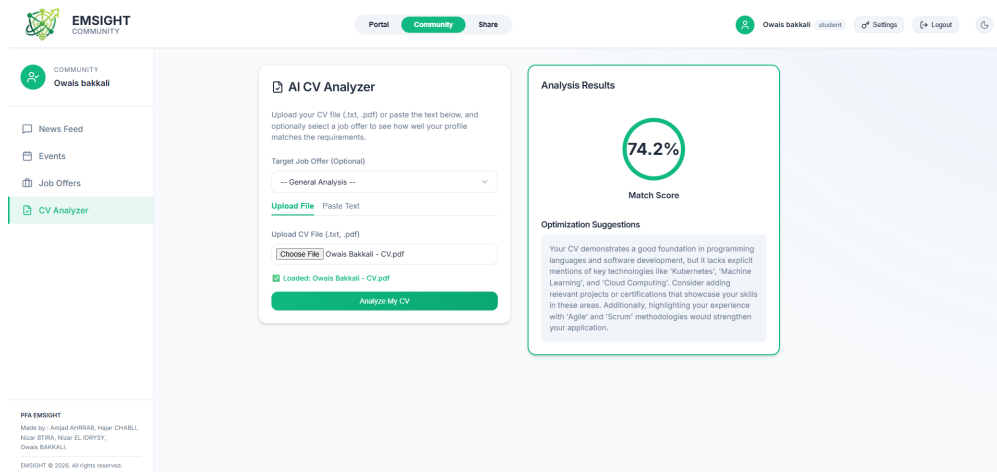


FIGURE 4.4 – Interface de l'analyseur de CV IA — score de correspondance et recommandations personnalisées

4.3.5 Dashboard administrateur — Gestion des emplois du temps

L'outil de construction des emplois du temps permet à l'administrateur d'ajouter des créneaux (jour, horaire, matière, enseignant) dans une grille interactive. Les données sont sérialisées en JSON et sauvegardées dans le modèle `ClassSchedule`, immédiatement visibles pour les étudiants et enseignants des classes concernées.

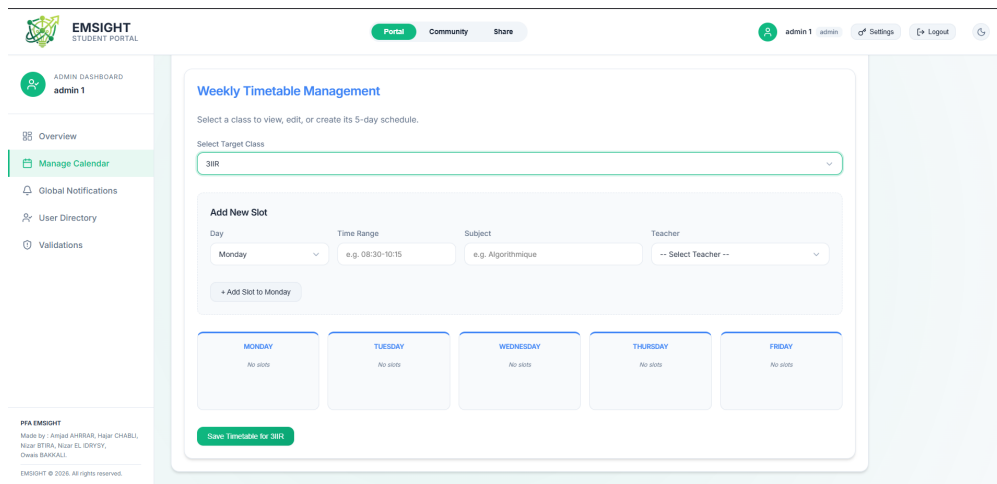


FIGURE 4.5 – Outil de gestion des emplois du temps — grille interactive hebdomadaire par classe

4.4 Extraits de code significatifs

4.4.1 Modèle CustomUser — accounts/models.py

```

1 from django.db import models
2 from django.contrib.auth.models import AbstractUser
3
4 class CustomUser(AbstractUser):
5     ROLE_CHOICES = (
6         ('student', 'Student'),
7         ('teacher', 'Teacher'),
8         ('admin', 'Administration'),
9     )
10    role = models.CharField(
11        max_length=20, choices=ROLE_CHOICES, default='student'
12    )

```

```
13     matricule = models.CharField(  
14         max_length=20, blank=True, null=True, unique=True  
15     )  
16     gender = models.CharField(  
17         max_length=10,  
18         choices=(( 'M', 'Male'), ( 'F', 'Female') ),  
19         default='M'  
20     )  
21     profile_picture = models.ImageField(  
22         upload_to='profile_pics/', blank=True, null=True  
23     )
```

Listing 4.1 – Modèle CustomUser — extension d'AbstractUser avec champ role

4.4.2 Soumission groupée des notes — portal/views.py

```
1 @action(detail=False, methods=[ 'post' ])  
2 def bulk_submit_grades(self, request):  
3     academic_year = request.data.get( 'academic_year', '2025-2026'  
4 )  
5     semester = request.data.get( 'semester', 'S1'  
6 )  
7     subject = request.data.get( 'subject'  
8 )  
9     evaluation_type = request.data.get( 'evaluation_type', 'CC'  
10 )  
11     grades_data = request.data.get( 'grades', [] )  
12  
13     for entry in grades_data:  
14         student_id = entry.get( 'student_id'  
15 )  
16         value = entry.get( 'value'  
17 )  
18         if student_id is None or value is None:  
19             continue  
20         rc, _ = ReportCard.objects.get_or_create(  
21             student_id=student_id,  
22             academic_year=academic_year,  
23             semester=semester  
24 )  
25         Grade.objects.update_or_create(  
26             report_card=rc, subject=subject,  
27             evaluation_type=evaluation_type,  
28             defaults={ 'value': float(value) }  
29 )  
30     all_grades = Grade.objects.filter(report_card=rc)
```

```

25     if all_grades.exists():
26         rc.general_average = round(
27             sum(g.value for g in all_grades) / all_grades.count
28             (), 2
29         )
30         rc.save()
31     return Response({'message': 'Grades submitted successfully'})

```

Listing 4.2 – Endpoint bulk_submit_grades — saisie en masse avec recalcul de moyenne

4.4.3 Routage par rôle — src/App.jsx

```

1  const renderContent = () => {
2      if (currentPlatform === 'community')
3          return <CommunityDashboard activeTab={activeTab} demoUser={user} />;
4      if (currentPlatform === 'share')
5          return <ShareDashboard activeTab={activeTab} demoUser={user} />;
6      switch (user.role) {
7          case 'student': return <StudentDashboard activeTab={activeTab} demoUser={user} />;
8          case 'teacher': return <TeacherDashboard activeTab={activeTab} demoUser={user} />;
9          case 'admin':   return <AdminDashboard activeTab={activeTab} demoUser={user} />;
10         default:        return <div>Role inconnu</div>;
11     }
12 };

```

Listing 4.3 – Routage conditionnel par rôle et plateforme — App.jsx (extrait)

Conclusion

Ce chapitre a présenté l'environnement technique d'EMSIGHT, les technologies choisies et leur justification, les implémentations clés (authentification RBAC, emplois du temps dynamiques, validation multi-enseignants, analyseur IA) ainsi que les interfaces réalisées. L'ensemble de ces éléments constitue une plateforme fonctionnelle,

robuste et maintenable, prête pour un déploiement en environnement de production.

Conclusion Générale

Bilan du projet

Le projet EMSIGHT représente une réponse concrète et techniquement solide à la problématique de fragmentation des outils numériques académiques à l'EMSI. En consolidant un portail académique (**EMSIGHT Portal**), une bibliothèque collaborative (**EMSI Share**) et un réseau professionnel avec analyse IA (**EMSI Community**) dans une interface unifiée, la plateforme offre une expérience utilisateur moderne et cohérente.

D'un point de vue technique, le projet a permis à l'équipe de concevoir et maîtriser une architecture décuplée client-serveur complète, depuis la modélisation UML et Merise jusqu'au déploiement d'une API REST sécurisée avec contrôle d'accès basé sur les rôles. L'intégration de l'intelligence artificielle pour l'analyse de CV, avec son mécanisme de repli heuristique, constitue une innovation pédagogique et technique au sein de l'écosystème EMSI.

Apports du projet

- **Unification** : remplacement de plusieurs environnements numériques distincts par un écosystème cohérent avec authentification unique.
- **Innovation pédagogique** : le système de validation multi-enseignants d'EMSI Share introduit une dynamique de revue par les pairs inédite à l'EMSI.
- **Aide à l'insertion** : l'analyseur de CV IA est une fonctionnalité sans équivalent dans l'écosystème numérique EMSI existant.
- **Robustesse** : la dégradation gracieuse (fallback heuristique), la séparation

des responsabilités et l'architecture modulaire garantissent la disponibilité et la maintenabilité de la plateforme.

- **Compétences acquises** : maîtrise de React 19, Django REST Framework, architecture REST, modélisation UML/Merise, intégration d'API IA, gestion de projet en équipe de cinq.

Perspectives

Plusieurs axes d'amélioration sont envisagés pour les versions futures de la plateforme :

- **Migration PostgreSQL** : déploiement en production avec un moteur de base de données adapté aux fortes charges.
- **Notifications temps réel** : intégration de WebSockets (Django Channels) pour des alertes instantanées push.
- **Application mobile** : portage de l'interface React en application mobile hybride via React Native.
- **Tableaux de bord analytiques** : visualisation des taux de présence, de la performance par filière et des tendances des offres professionnelles.
- **Internationalisation complète** : activation du module i18next (déjà intégré) pour l'arabe et l'anglais.

Bibliographie

- [1] Moodle HQ, “Moodle — Open-source Learning Management System.” <https://moodle.org>, 2024.
- [2] T. Christie *et al.*, “Django REST Framework Documentation.” <https://www.django-rest-framework.org>, 2025. Version 3.17.
- [3] Meta Platforms, Inc., “React Documentation.” <https://react.dev>, 2025. Version 19.
- [4] E. You, “Vite — Next Generation Frontend Tooling.” <https://vitejs.dev>, 2025. Version 8.0.
- [5] M. Zabriskie *et al.*, “Axios — Promise based HTTP client for the browser and node.js.” <https://axios-http.com>, 2024. Version 1.15.
- [6] Groq Inc., “Groq API Documentation — Llama 3.3 70B Versatile.” <https://console.groq.com/docs>, 2025.
- [7] Mozilla Foundation, “PDF.js — A Portable Document Format (PDF) viewer.” <https://mozilla.github.io/pdf.js>, 2025. Version 5.7.

Annexe A

Extraits de Code Significatifs

A.1 Backend – Modèle CustomUser (accounts/models.py)

Le modèle CustomUser ci-dessous étend AbstractUser de Django. Le champ role détermine l'ensemble des droits d'accès de l'utilisateur au sein de la plateforme EMSIGHT.

```
1 from django.db import models
2 from django.contrib.auth.models import AbstractUser
3
4 class CustomUser(AbstractUser):
5     ROLE_CHOICES = (
6         ('student', 'Student'),
7         ('teacher', 'Teacher'),
8         ('admin', 'Administration'),
9     )
10    role = models.CharField(
11        max_length=20, choices=ROLE_CHOICES, default='student'
12    )
13    matricule = models.CharField(
14        max_length=20, blank=True, null=True, unique=True
15    )
16    gender = models.CharField(
17        max_length=10,
18        choices=(('M', 'Male'), ('F', 'Female')),
19        default='M'
20    )
```

```
21     profile_picture = models.ImageField(  
22         upload_to='profile_pics/', blank=True, null=True  
23     )  
24  
25     def __str__(self):  
26         return f"{self.first_name} {self.last_name} ({self.role})"
```

Listing A.1 – Modele CustomUser – accounts/models.py

A.2 Backend – Endpoint de Connexion (accounts/views.py)

L'action login expose l'endpoint POST /api/accounts/users/login/. Elle retrouve l'utilisateur par email, appelle `authenticate()` avec le nom d'utilisateur Django correspondant, et retourne les données sérialisées ou un message d'erreur.

```
1 @action(detail=False, methods=['post'])  
2 def login(self, request):  
3     from django.contrib.auth import authenticate  
4     email = request.data.get('email')  
5     password = request.data.get('password')  
6  
7     if not email or not password:  
8         return Response(  
9             {'error': 'Please provide email and password'},  
10            status=400  
11        )  
12  
13     try:  
14         user_obj = CustomUser.objects.get(email=email)  
15     except CustomUser.DoesNotExist:  
16         return Response({'error': 'Invalid credentials'}, status  
17            =401)  
18  
19     user = authenticate(username=user_obj.username, password=  
20        password)  
21     if user is not None:  
22         return Response(UserSerializer(user).data)  
23     else:  
24         return Response({'error': 'Invalid credentials'}, status  
25            =401)
```

Listing A.2 – Action login – accounts/views.py

A.3 Backend – Service Analyse CV avec Fallback (community/ai_service.py)

L'extrait ci-dessous illustre la logique principale du service d'analyse de CV : tentative d'appel à l'API Groq, et en cas d'échec, basculement automatique vers l'algorithme heuristique de correspondance par mots-clés.

```
1 def analyze_cv(cv_text, job_requirements):
2     """
3     Analyse locale du CV par correspondance de mots-clés.
4     Utilisee comme fallback si l'API Groq est indisponible.
5     """
6     req_keywords = _extract_keywords(job_requirements)
7     cv_keywords = _extract_keywords(cv_text)
8
9     if not req_keywords:
10         return 0, "Job requirements are too vague to analyze."
11     if not cv_keywords:
12         return 0, "Your CV does not contain recognizable keywords."
13
14     matched = req_keywords & cv_keywords
15     missing = req_keywords - cv_keywords
16
17     score = round(len(matched) / len(req_keywords) * 100, 1)
18
19     lines = ["Analysis results (keyword matching):"]
20     if score >= 75:
21         lines.append(f"Great match! {len(matched)}/{len(
22 req_keywords)} keywords found.")
23     elif score >= 40:
24         lines.append(f"Decent match: {len(matched)}/{len(
25 req_keywords)} keywords.")
26     else:
27         lines.append(f"Low match: {len(matched)}/{len(req_keywords)
28 } keywords.")
```

```
27     if matched:
28         lines.append(f"Matched: {'', '.join(sorted(matched)[:8])}.".")
29     if missing:
30         lines.append(f"Missing: {'', '.join(sorted(missing)[:8])}.".")
31
32     return score, "\n".join(lines)
```

Listing A.3 – Algorithme heuristique de fallback – ai_service.py

A.4 Frontend – Client Axios Centralisé (src/api.js)

```
1 import axios from 'axios';
2
3 const api = axios.create({
4     baseURL: 'http://127.0.0.1:8000/api/',
5 });
6
7 export default api;
```

Listing A.4 – Client Axios centralise – src/api.js

A.5 Frontend – Routage par Rôle (src/App.jsx, extrait)

```
1 const renderContent = () => {
2     if (currentPlatform === 'community') {
3         return <CommunityDashboard activeTab={activeTab} demoUser={user}
4         />;
5     }
6     if (currentPlatform === 'share') {
7         return <ShareDashboard activeTab={activeTab} demoUser={user}
8         />;
9     }
10
11     switch (user.role) {
12         case 'student':
13             return <StudentDashboard activeTab={activeTab} demoUser={user}
14             />;
15         case 'teacher':
```

```
13     return <TeacherDashboard activeTab={activeTab} demoUser={user  
14   } />;  
15   case 'admin':  
16     return <AdminDashboard activeTab={activeTab} demoUser={user}  
17   />;  
18   default:  
19     return <div>Role inconnu</div>;  
20 }  
21 };
```

Listing A.5 – Routage conditionnel par role et plateforme – App.jsx

A.6 Backend – Soumission Groupée des Notes (portal/views.py, extrait)

```
1 @action(detail=False, methods=['post'])  
2 def bulk_submit_grades(self, request):  
3     academic_year = request.data.get('academic_year', '2025-2026')  
4     semester      = request.data.get('semester', 'S1')  
5     subject       = request.data.get('subject')  
6     evaluation_type = request.data.get('evaluation_type', 'CC')  
7     grades_data    = request.data.get('grades', [])  
8  
9     for grade_entry in grades_data:  
10         student_id = grade_entry.get('student_id')  
11         value = grade_entry.get('value')  
12         if student_id is None or value is None:  
13             continue  
14  
15         rc, _ = ReportCard.objects.get_or_create(  
16             student_id=student_id,  
17             academic_year=academic_year,  
18             semester=semester  
19         )  
20         Grade.objects.update_or_create(  
21             report_card=rc,  
22             subject=subject,  
23             evaluation_type=evaluation_type,  
24             defaults={'value': float(value)}  
25         )
```

```
26
27     all_grades = Grade.objects.filter(report_card=rc)
28     if all_grades.exists():
29         rc.general_average = round(
30             sum(g.value for g in all_grades) / all_grades.count
31             ), 2
32         rc.save()
33
34     return Response({'message': 'Grades submitted successfully'})
```

Listing A.6 – Soumission groupée de notes – portal/views.py

Annexe B

Captures d'Écran Complémentaires

B.1 Module Community – Fil d'Actualité

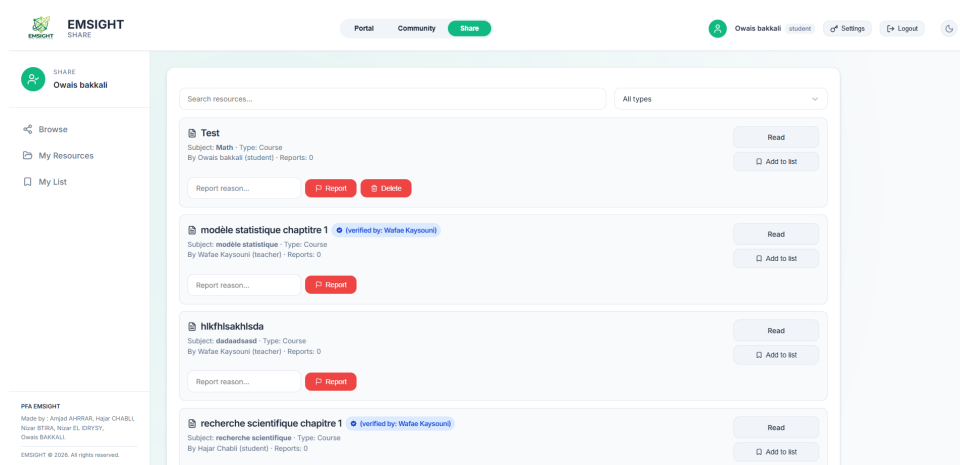


FIGURE B.1 – Fil d'actualité EMSI Community : publication de posts avec médias, badges de rôle (Admin, Teacher) et système de validation par les enseignants

B.2 Dashboard Professeur – Saisie des Notes

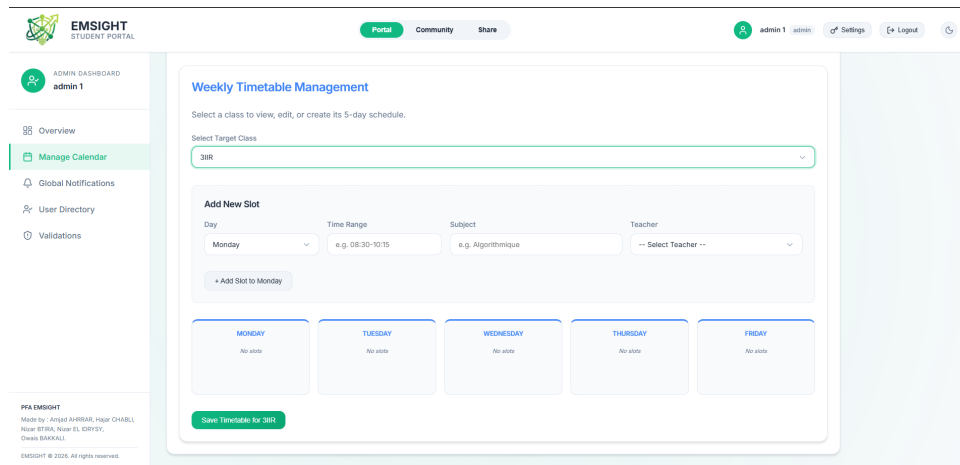


FIGURE B.2 – Interface de saisie des notes par classe – Dashboard Professeur : sélection du module, de la classe, du semestre et du type d'évaluation, puis saisie individuelle avec soumission groupée

B.3 Bulletin Étudiant – Vue Transcript Global

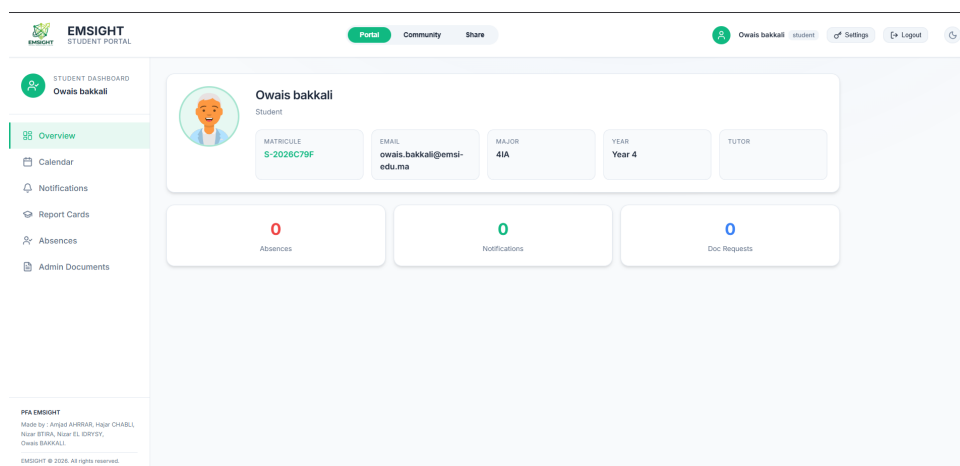


FIGURE B.3 – Relevé de notes global (Transcript) – Dashboard Étudiant : vue multi-années avec détail par semestre, moyenne générale, statut de validation et rattrapages